

TP numéro 6

Gestion des processus et services

Prérequis et environnement

Vous allez travailler dans la machine virtuelle **Ubuntu Server**.

Exercice 1

L'objectif est d'observer la différence de comportement entre un arrêt propre et un arrêt brutal.

1. Lancez une commande longue en arrière-plan (ex : `sleep 1000 &`).
2. Identifiez son PID à l'aide de la commande `ps` et vérifiez son état dans `/proc/[PID]/status`.
3. **Expérimentation SIGTERM :**

— Créez un script nommé `handler_demo.sh` avec le contenu suivant :

```
1  #!/bin/bash
2
3  # routine de fermeture
4  cleanup() {
5      echo "[$(date)] Nettoyage avant sortie..." >> /tmp/demo.log
6      exit 0
7  }
8
9  #trap : commande shell qui intercepte des signaux ou événements
   système
10 trap cleanup SIGTERM
11
12 echo "PID du script : $$"
13 while true; do
14     echo "En cours d'exécution..."
15     sleep 5
16 done
```

— Envoyez un SIGTERM (15) et observez si le processus peut exécuter la routine de fermeture (`kill -15 <PID>`).

4. **Expérimentation SIGKILL :**

— Relancez le processus et envoyez un SIGKILL (9) (`kill -9 <PID>`).

— Pourquoi est-il impossible pour le processus d'intercepter ce signal ? Quelles sont les conséquences potentielles sur une base de données en production ?

Exercice 2

1. Lancez `sleep 300` en arrière-plan avec `nohup` : `nohup sleep 300 &`

2. Fermez le terminal et ouvrez-en un nouveau. Vérifiez si `sleep` est toujours actif avec `ps aux | grep sleep`. Notez votre observation.

Exercice 3

1. Créez un script `/usr/local/bin/gi_ia_service.sh` avec le contenu suivant :

```
1 #!/bin/bash
2 while true; do
3     echo "Service actif à $(date)" >> /var/tmp/gi_ia_service.log
4     sleep 10
5 done
```

Rendez-le exécutable.

2. Créez un fichier de service systemd `/etc/systemd/system/gi_ia_service.service` (avec `sudo`) contenant :

```
1 [Unit]
2 Description=Mon service de démonstration
3 After=network.target
4
5 [Service]
6 Type=simple
7 ExecStart=/usr/local/bin/gi_ia_service.sh
8 Restart=on-failure
9
10 # Sandboxing
11 User=nobody
12 Group=nogroup
13 ProtectSystem=strict
14 PrivateTmp=true
15 NoNewPrivileges=true
16 ProtectHome=true
17
18
19 [Install]
20 WantedBy=multi-user.target
```

3. Activez et démarrez le service :

```
1 sudo systemctl daemon-reload
2 sudo systemctl enable gi_ia_service.service
3 sudo systemctl start gi_ia_service.service
```

4. Vérifiez que le service fonctionne : `systemctl status gi_ia_service`. Quel est l'état ? Consultez le fichier de log `/var/tmp/gi_ia_service.log`.
5. **Test du sandboxing** : Essayez d'accéder au répertoire `/home` depuis l'intérieur du service. Pour cela, modifiez temporairement le script pour y inclure `ls /home` et redémarrez le service. Que se passe-t-il ? (Indice : consultez les logs avec `journalctl -u gi_ia_service`).
6. Ajoutez une limitation de mémoire à 50 Mo dans la section `[Service]` : `MemoryMax=50M`. Redémarrez le service. Vérifiez avec `systemctl show gi_ia_service | grep MemoryCurrent` (peut nécessiter quelques secondes). Que se passe-t-il si le service dépasse cette limite ?

Exercice 4 (optionnel)

On souhaite automatiser une sauvegarde du répertoire `/etc` vers `/var/backups/system`.

1. **Méthode Cron** : Configurez une tâche `cron` pour l'utilisateur `root` qui effectue cette sauvegarde chaque lundi à 03h00. Utilisez des chemins absolus.
2. **Méthode Systemd Timer** :
 - Créez un script `/usr/local/bin/backup_etc.sh` (pensez au `chmod +x`).
 - Créez un fichier unité `backup-etc.service` de type `oneshot`.
 - Créez le fichier `backup-etc.timer` associé pour une exécution hebdomadaire.
3. **Analyse comparative** : Redémarrez votre machine. Expliquez comment la directive `Persistent=true` du timer permet de rattraper la sauvegarde manquée, contrairement à `cron`.